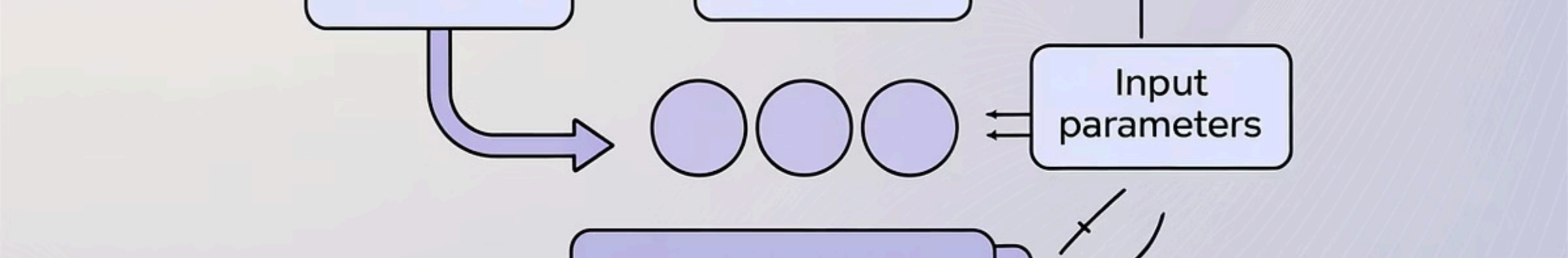




Funciones en JavaScript

Domina el arte de escribir código limpio y reutilizable con funciones JavaScript. En este módulo aprenderás las diferentes formas de crear funciones, sus características únicas, y cómo utilizarlas de manera efectiva en tus proyectos.



¿Qué son las Funciones?



Bloque de Código

Las funciones son contenedores que agrupan instrucciones relacionadas para realizar una tarea específica.



Reutilización

Una vez creadas, puedes invocar las funciones tantas veces como necesites sin repetir código.



Organización

Dividen tu código en partes manejables y comprensibles, mejorando la estructura del proyecto.

En JavaScript existen múltiples maneras de crear funciones, cada una con sus propias ventajas y casos de uso. Exploraremos las tres formas principales: declarativas, expresadas y arrow functions.

Funciones Declarativas

Las funciones declarativas son la forma clásica de crear funciones en JavaScript. Se definen utilizando la palabra clave `function` seguida de un nombre obligatorio.

```
function saludar() {  
  console.log("¡Hola mundo!");  
}  
  
saludar(); // Invoca la función
```

Nombre Obligatorio

Siempre deben tener un identificador único para poder ser invocadas.

Hoisting

Pueden ser llamadas antes de su declaración gracias al "elevado" de JavaScript.



Ejemplo Práctico: Función Declarativa

01

Definición

Creamos una función que calcula el área de un rectángulo

03

Procesamiento

Multiplica los valores y devuelve el resultado

02

Parámetros

La función recibe dos valores: ancho y alto

04

Invocación

Llamamos la función con valores específicos

```
// Definición de la función
function calcularArea(ancho, alto) {
  const area = ancho * alto;
  return area;
}
```

```
// Invocación - puede hacerse antes de la declaración
console.log(calcularArea(5, 3)); // 15
console.log(calcularArea(8, 4)); // 32
```

 **Ventaja del Hoisting:** Puedes organizar tu código poniendo las funciones al final del archivo y las invocaciones al principio, mejorando la legibilidad.

Funciones Expresadas

Las funciones expresadas se almacenan en variables, convirtiéndose en valores asignables. Esta técnica ofrece mayor flexibilidad en la estructura del código.

Sintaxis Básica

```
const despedir = function() {  
  console.log("¡Hasta luego!");  
};
```

```
despedir(); // Invocación
```

Con Nombre (Opcional)

```
const saludar = function saludo() {  
  console.log("¡Hola!");  
};
```

```
saludar(); // Funciona
```

Sin Hoisting

Deben declararse antes de ser utilizadas, evitando comportamientos inesperados.

Nombre Opcional

Pueden ser anónimas o tener un identificador interno para depuración.

Asignación Dinámica

Pueden reasignarse a diferentes funciones durante la ejecución.



Arrow Functions (Funciones Flecha)

ES6 Revolution

Las arrow functions introdujeron una sintaxis más concisa y elegante en ES6. Son especialmente útiles para funciones cortas y callbacks, ofreciendo un código más limpio y legible.

Sintaxis Completa

1

```
const sumar = (a, b) => {  
  return a + b;  
};
```

2

Sintaxis Corta

```
const sumar = (a, b) => a + b;
```

Un Parámetro

3

```
const cuadrado = n => n * n;
```

4

Sin Parámetros

```
const obtenerFecha = () => new Date();
```

Ventajas de las Arrow Functions



Concisión

Reduce significativamente la cantidad de código necesario para funciones simples.



this Léxico

Mantiene el contexto de `this` del ámbito circundante, evitando confusiones.



Legibilidad

Especialmente útiles en operaciones con arrays como `map`, `filter` y `reduce`.

Ejemplo Comparativo

Función Tradicional

```
const numeros = [1, 2, 3, 4, 5];
const dobles = numeros.map(function(n) {
  return n * 2;
});
```

Arrow Function

```
const numeros = [1, 2, 3, 4, 5];
const dobles = numeros.map(n => n * 2);
```

Parámetros por Defecto

Los parámetros por defecto permiten que las funciones tengan valores predefinidos cuando no se pasan argumentos específicos. Esta característica hace que las funciones sean más robustas y flexibles.



Configuración Inicial

Define valores predeterminados directamente en la declaración de parámetros.



Protección contra Errores

Evita errores por valores `undefined` en parámetros no proporcionados.



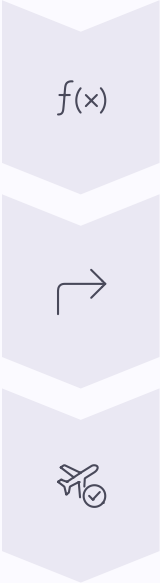
Flexibilidad de Uso

Permite llamar funciones con diferentes cantidades de argumentos.

```
function crearUsuario(nombre = "Invitado", edad = 18, activo = true) {  
  return {  
    nombre: nombre,  
    edad: edad,  
    activo: activo  
  };  
}  
  
console.log(crearUsuario());  
// { nombre: "Invitado", edad: 18, activo: true }  
  
console.log(crearUsuario("Ana", 25));  
// { nombre: "Ana", edad: 25, activo: true }
```

Callbacks: Funciones como Argumentos

Los callbacks son funciones que se pasan como argumentos a otras funciones. Este patrón es fundamental en JavaScript para manejar operaciones asíncronas y crear código más modular y flexible.



$f(x)$

Función Principal

Recibe otra función como parámetro



Callback

Función que se ejecuta en un momento específico



Resultado

Comportamiento personalizable y reutilizable

Ejemplo Práctico

```
function procesarDatos(datos, callback) {  
  console.log("Procesando...");  
  const resultado = datos.map(x => x * 2);  
  callback(resultado);  
}  
  
// Diferentes callbacks para diferentes comportamientos  
procesarDatos([1, 2, 3], (resultado) => {  
  console.log("Resultado:", resultado); // [2, 4, 6]  
});  
  
procesarDatos([1, 2, 3], (resultado) => {  
  console.log("Suma total:", resultado.reduce((a, b) => a + b));  
});
```

Resumen y Mejores Prácticas

Declarativas

Cuándo usar: Funciones principales del programa que necesites llamar antes de declararlas.

Ventaja: Hoisting para mayor flexibilidad organizativa.

Expresadas

Cuándo usar: Funciones que necesites asignar condicionalmente o pasar como valores.

Ventaja: Mayor control sobre cuándo se crean.

Arrow Functions

Cuándo usar: Callbacks, funciones cortas y cuando necesites preservar el contexto de `this`.

Ventaja: Sintaxis concisa y comportamiento predecible.



Consejos Clave

Usa nombres descriptivos

Los nombres de funciones deben explicar claramente qué hacen: `calcularImpuestos()` es mejor que `calc()`.

Mantén funciones pequeñas

Una función debería hacer una sola cosa bien. Si es muy larga, probablemente necesita dividirse.

Aprovecha los parámetros por defecto

Hacen tus funciones más robustas y fáciles de usar, reduciendo la necesidad de validaciones manuales.

✅ **¡Próximo paso!** Practica creando funciones de cada tipo y experimenta con callbacks. La comprensión profunda de las funciones es clave para dominar JavaScript avanzado.